

retrieval-augmented generation

artificial intelligence (AI) framework

Retrieval

Key notes

Stage 1 (Foundations / Recap)

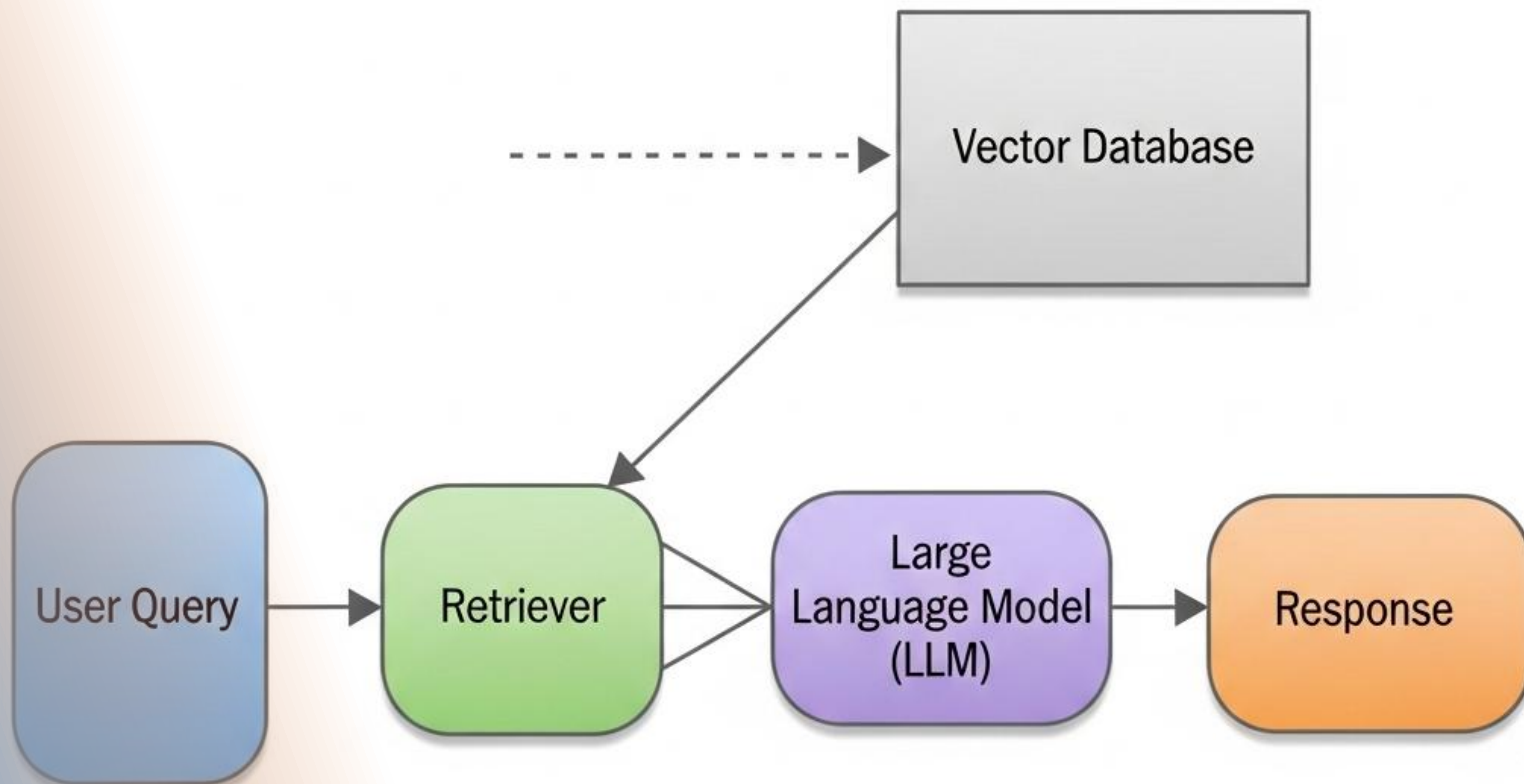
the **basics of RAG** before introducing advanced variants.

Clarify the **retriever–generator loop** and its limitations.

Highlight **why “Advanced RAG” is needed** (hallucinations, recall issues, personalization gaps).

What is RAG?

- **Retrieval-Augmented Generation (RAG)** is an architecture that combines:
 - **Retriever** → finds relevant information from an external knowledge source (e.g., vector database, document store).
 - **Generator (LLM)** → uses both the retrieved documents and the user query to produce a response.
- 👉 Motivation: LLMs have limited context windows and may hallucinate. RAG grounds their responses in external knowledge.



Simple RAG

- A user query is received.
- The query is sent to a Retriever, which searches a vast, external Vector Database (your knowledge base) for relevant documents or text chunks.
- The most relevant documents are retrieved and then combined with the original user query to create an augmented prompt.
- This new, richer prompt is then passed to a Large Language Model (LLM).
- The LLM generates a final, contextually informed response based on both the original query and the retrieved information.



Types of Retrievers

- **Sparse Retrieval (BM25, TF-IDF):**
 - Matches keywords between query and documents.
 - Pros: Simple, interpretable.
 - Cons: Misses semantic meaning.
- **Dense Retrieval (Embeddings, DPR, OpenAI embeddings):**
 - Uses neural embeddings for semantic similarity.
 - Pros: Captures meaning beyond keywords.
 - Cons: Requires more compute + ANN indexing.
- **Hybrid Retrieval:**
 - Combines sparse + dense approaches.
 - Pros: Best of both worlds (semantic + exact match).
 - Cons: More complex to implement.

Keyword / Lexical Search (BM25, TF-IDF)

Description

- Matches **query keywords** with document terms.
- No embeddings, purely statistical scoring.

LLM Use Case

- **Exact phrase search** in legal, medical, or scientific documents.
- Acts as a **fallback retriever** in RAG pipeline

Usage

Examples

- Query: "COVID-19 treatment"
- Matches documents containing these exact keywords or close variants.

Libraries/Tools

- **ElasticSearch, Apache Lucene, Whoosh, Solr.**

Use Cases

- Enterprise document search
- Legal or compliance search
- Baseline for hybrid RAG systems

Example code

```
from rank_bm25 import BM25Okapi

docs = [
    "Vitamin C reduces common cold duration.",
    "Quantum entanglement does not allow faster-than-light communication.",
    "Artificial sweeteners are safe for human consumption."
]
tokenized_docs = [doc.split(" ") for doc in docs]
bm25 = BM25Okapi(tokenized_docs)

query = "cold treatment vitamin C"
results = bm25.get_top_n(query.split(" "), docs, n=2)

print("BM25 Results:", results)
```

Viewpoints

Pros

- Simple, interpretable, fast.
- No embeddings or heavy models needed.
- Deterministic results.

✗ Cons

- Ignores semantics (e.g., synonyms).
- Poor results for paraphrased queries.
- Limited to vocabulary overlap.

Dense Retrieval (Embeddings, DPR, OpenAI embeddings)

Uses neural embeddings for semantic similarity.

Pros: Captures meaning beyond keywords.

Cons: Requires more compute + ANN indexing.

Dense Vector Retrieval (Embeddings)

Description

- **Encodes** text into dense vector embeddings.
- Retrieves documents by computing cosine similarity between query and document embeddings.

Examples

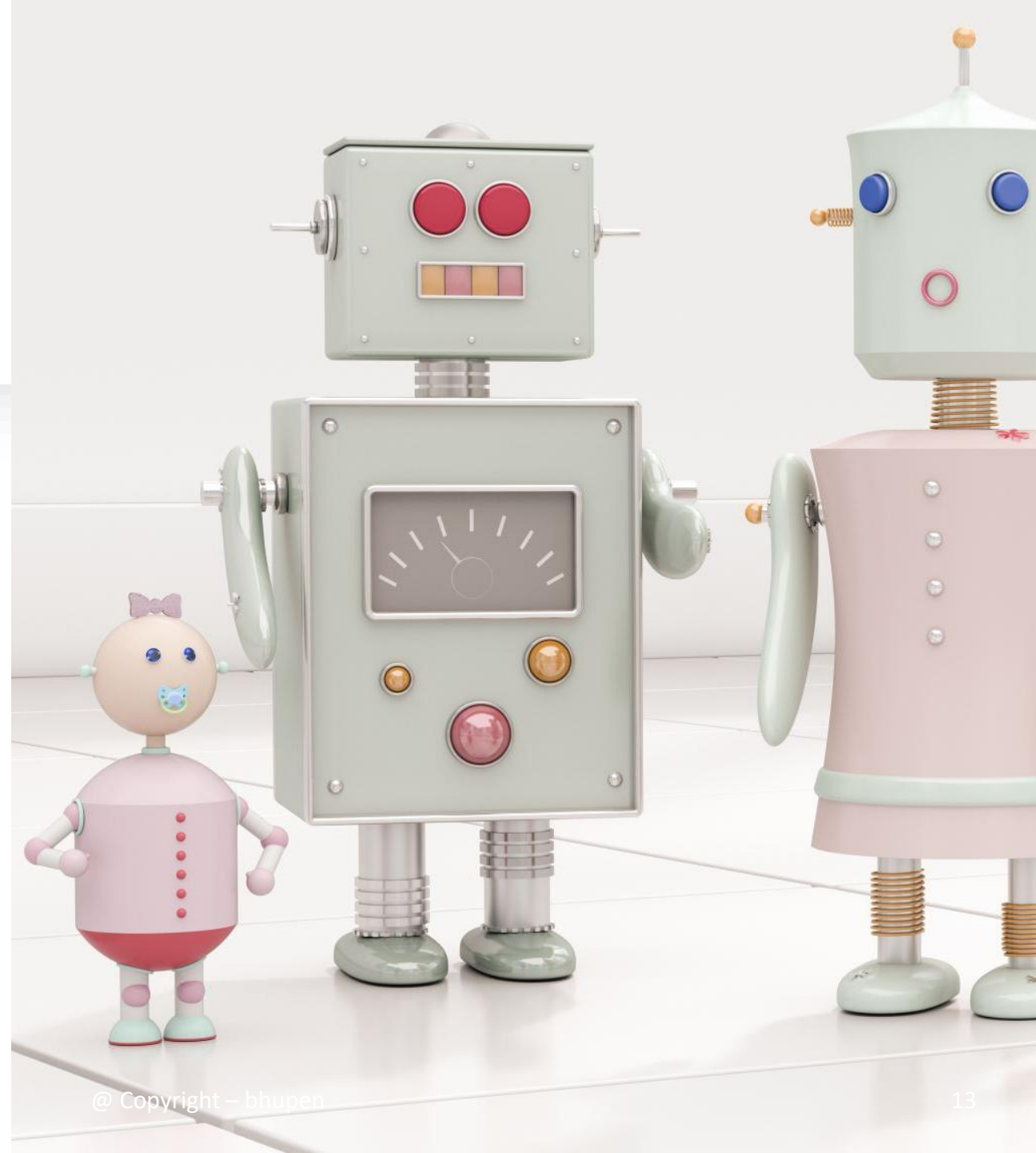
- Query: "flu medicine"
- Matches "influenza treatment" even without keyword overlap.

Libraries/Tools

- OpenAI Embeddings API (text-embedding-3-large)
- Sentence-BERT
- FAISS, ChromaDB, Weaviate

Use Cases

- **RAG** for chatbots and scientific Q&A.
- Semantic search in large unstructured knowledge bases.



Python Code (Demo)

```
from openai import OpenAI
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

client = OpenAI()

docs = ["Vitamin C for cold recovery",
        "COVID-19 treatments available",
        "Quantum physics insights"]

# Embed docs
embeddings = [client.embeddings.create(model="text-embedding-3-small",
                                       input=doc).data[0].embedding for doc in docs]

# Query embedding
query = "cold treatment"
query_emb = client.embeddings.create(model="text-embedding-3-small",
                                     input=query).data[0].embedding

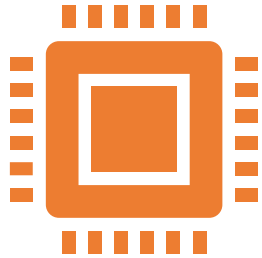
# Compute similarity
scores = cosine_similarity([query_emb], embeddings)
top_idx = np.argsort(scores[0])[::-1][:2]

print(" ♦ Dense Retrieval:", [docs[i] for i in top_idx])
```

Approximate Nearest Neighbor (ANN) Search

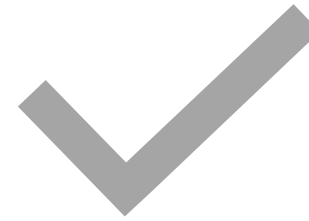
Description	Optimizes large-scale vector similarity search using approximate algorithms (e.g., HNSW, IVF, PQ) for speed.
Examples	Retrieve top 100 relevant documents from a billion-record dataset in milliseconds.
Libraries/ Tools	FAISS (HNSW, IVF, PQ) Annoy Milvus, Weaviate
Use Cases	Scaling RAG systems with millions/billions of documents. Real-time search in chat assistants.

Viewpoints



✓ Pros

Fast and memory-efficient.
Scales well to massive datasets.



✗ Cons

Results approximate (may miss some matches).
Needs tuning for accuracy vs. speed.

Python Code (Demo)

```
import faiss
import numpy as np

# Random embeddings (simulate LLM embeddings)
vectors = np.random.rand(1000, 128).astype('float32')

# Build ANN index
index = faiss.IndexHNSWFlat(128, 32)
index.add(vectors)

query = np.random.rand(1, 128).astype('float32')
D, I = index.search(query, k=5)
print(" ♦ ANN Retrieved indices:", I)
```

Hybrid Retrieval (Keyword + Dense)

Hybrid retrieval **combines the strengths of keyword-based retrieval (BM25, TF-IDF) and semantic retrieval (dense embeddings)** to improve the overall search quality in LLM-based applications.

Keyword search ensures **exact term matching**, which is crucial for names, IDs, acronyms, or legal phrases.

Dense embeddings capture **semantic meaning**, allowing retrieval of documents even when they don't share exact words with the query.

The final ranking is usually computed by **merging scores** from both methods (weighted sum, max score, or learning-to-rank).

Example Scenario

Query: "treatment for the flu"

Candidate Documents:

1. "Vitamin C for cold recovery" → **dense match (semantic)**
2. "Influenza treatment guidelines" → **keyword match (BM25)**
3. "Antiviral medicines for influenza" → **both keyword + semantic match**

Hybrid retrieval **fetches both #2 and #3** and ranks them higher than keyword-only or embedding-only approaches.

Libraries / Tools

- **LangChain:** MultiQueryRetriever, EnsembleRetriever
- **Vespa AI** (native hybrid search support)
- **ElasticSearch + FAISS** combination
- **Weaviate, OpenSearch** hybrid pipelines

Use Cases in LLM Applications

Legal/Regulatory AI Assistants:

- Keywords like “Article 21” must match exactly, but also retrieve related laws semantically.

Enterprise RAG:

- Internal documentation where acronyms and abbreviations are important.

Biomedical Q&A:

- Retrieve “influenza therapy” for query “flu treatment”.

Hybrid search

✓ Pros

- Best of both worlds: Recovers exact matches and semantic matches.
- Higher recall and precision.
- Works well for technical domains with specific terms.

✗ Cons

- Complex setup: Two retrievers + fusion logic.
- Score normalization needed (BM25 vs cosine similarity scales differ).
- More compute overhead.

Python code

```
from rank_bm25 import BM25Okapi
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Corpus
docs = [
    "Vitamin C for cold recovery",
    "Influenza treatment guidelines",
    "Antiviral medicines for influenza",
    "Quantum physics and time dilation"
]
```

```
# ----- BM25 Keyword Search -----
tokenized_docs = [doc.lower().split() for doc in docs]
bm25 = BM25Okapi(tokenized_docs)
query = "treatment for the flu"
bm25_scores = bm25.get_scores(query.lower().split())

# ----- Dense Embedding Search (simulated embeddings) -----
# Normally use OpenAI/Sentence-BERT embeddings
np.random.seed(42)
embeddings = np.random.rand(len(docs), 5)
query_emb = np.random.rand(1, 5)
dense_scores = cosine_similarity(query_emb, embeddings)[0]
```

```
# ----- Combine scores -----
alpha = 0.5 # weighting factor
final_scores = alpha * bm25_scores + (1 - alpha) * dense_scores
top_idx = np.argsort(final_scores)[::-1][:3]

print(" ♦ Hybrid Retrieval Results:")
for i in top_idx:
    print(f"- {docs[i]} (score={final_scores[i]:.3f})")
```

Bi-Encoder Retrieval

Bi-Encoder Retrieval is a dual-encoder architecture where:

A **query encoder** converts the user question into a fixed-size dense embedding vector.

A **document encoder** converts each document in the corpus into an embedding vector.

Then,

a similarity search (cosine or dot product) is performed to retrieve the most relevant documents for a given query.

The key idea:

Embeddings are computed separately (hence "bi-encoder").

Documents can be encoded **offline once and stored** in a vector database, making retrieval **fast and scalable**.

Architecture

Query → [Query Encoder] → 768-d vector

|

|

Cosine Similarity



Docs → [Doc Encoder] → 768-d vector

Retrieve Top-k most similar documents



Example Flow

Input:

Query:

"best medicine for flu"

Docs:

"Flu treated with antivirals"

"Quantum computing basics"

"Vitamin C helps with cold symptoms"

Steps:

Encode query → Vector Q

Encode docs → Vectors D1, D2, D3

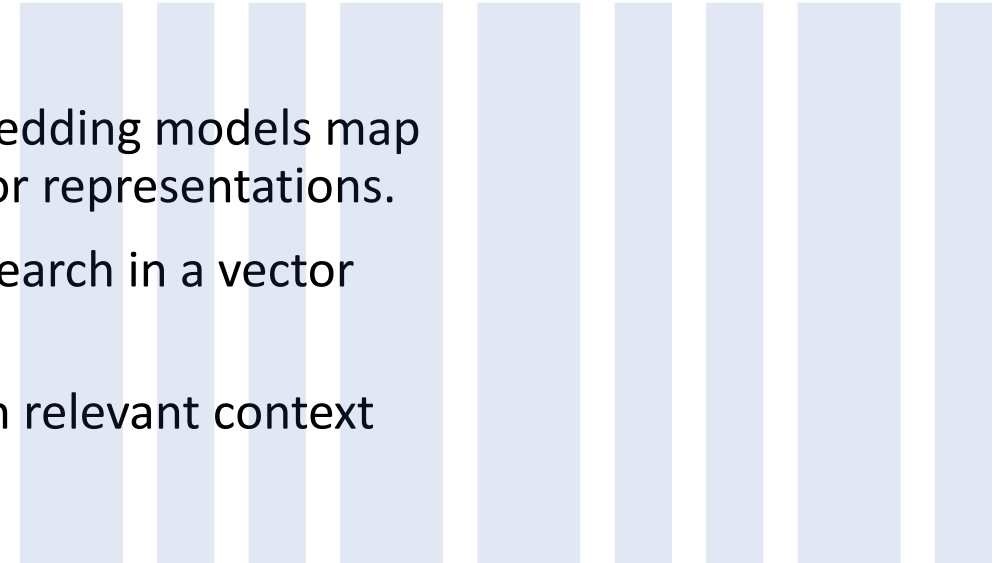
Compute similarity(Q, D_i)

Retrieve highest scores → Docs 1 and 3.

Common Ground Between Bi-Encoder and OpenAI Embeddings



- Both Bi-Encoder models and OpenAI embedding models map text (query or document) into dense vector representations.
- Both allow you to do semantic similarity search in a vector database like FAISS, Chroma, or Weaviate.
- Both can be used in RAG pipelines to fetch relevant context for LLMs.



Key Differences

Feature	Bi-Encoder (Custom / HuggingFace)	OpenAI Embeddings (text-embedding-3-large)
Architecture	Dual-Transformer encoders (usually Sentence-BERT variants)	Pretrained, large transformer (proprietary OpenAI)
Training	Often trained/fine-tuned on domain-specific QA pairs or similarity datasets	Trained on a massive, general-purpose corpus
Hosting	Runs locally or on your infra (HuggingFace, SentenceTransformers)	Hosted by OpenAI API (no local weights)
Customization	✅ You can fine-tune on your data (e.g., biomedical, legal text)	❌ No fine-tuning available (fixed embeddings)
Performance	Depends on model and fine-tuning quality	High quality general-purpose embeddings out of the box
Cost	Free once you have the model (just compute cost)	Pay-per-usage (tokens embedded)
Latency	Usually faster after setup (local GPU or batch encoding)	API call overhead for every new text
Control	Full transparency and modifiable architecture	Black-box, proprietary

Using a Custom Bi-Encoder

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

doc_emb = model.encode("Flu treated with antivirals")
query_emb = model.encode("best medicine for influenza")
```

Using OpenAI Embeddings

```
from openai import OpenAI
client = OpenAI()

embedding = client.embeddings.create(
    model = "text-embedding-3-large",
    input = "best medicine for influenza"
)
print(len(embedding.data[0].embedding))  # 3072 dimensions
```

Bi-Encoder Training: The Core Idea

The model is trained on **pairs of text**:

- **Positive pairs:** (query, relevant_document)
- **Negative pairs:** (query, irrelevant_document)

Goal:

- **Maximize similarity** (e.g., cosine similarity) for positive pairs.
- **Minimize similarity** for negative pairs.

This is typically achieved using **contrastive learning** or **triplet loss**.

Training Dataset

- **MS MARCO** (web queries + passages).
- **NQ (Natural Questions)** (questions + answers).
- **Domain-specific datasets:**
 - Biomedical: PubMedQA, BioASQ.
 - Legal: CaseLaw, Contract NLI datasets.

Fine-Tuning a Bi-Encoder for Domain-Specific LLM Apps

- Start from a **pretrained general model** (e.g., all-MiniLM-L6-v2).
- Fine-tune using **your claims dataset** and **corpus JSON**:
 - Claim = query.
 - Supporting abstracts = positive docs.
 - Random or BM25 retrieved irrelevant docs = negatives.
- Result: A **bi-encoder specialized in scientific claim verification** for your project.

View points

- **Pros**
 - **Domain-aware retrieval** → reduces irrelevant context for LLM.
 - **Better accuracy than generic embeddings** (like OpenAI's).
 - **Offline retrieval** → cheaper than API-based embeddings.
 - Can be used in **hybrid search** with BM25.
- **Cons**
 - Needs labeled data (query-positive pairs).
 - Compute-intensive to train from scratch (often fine-tuning is used).
 - Quality limited by training data diversity.

Cross-Encoder Reranking

- A **cross-encoder** takes a **query and document together as input**, encoding them jointly and outputting a **single relevance score**.
- Unlike bi-encoder, it **does not precompute document embeddings**, making it **more precise but slower**.
- It's typically used **after initial retrieval** (e.g., BM25 or Bi-Encoder) to **re-rank the top-k candidates** before passing them to an LLM.



Architecture

Query: "Who discovered penicillin?"

Doc1: "Alexander Fleming discovered penicillin in 1928."

Doc2: "Medical breakthroughs changed history."

For each (query, doc) pair:

[CLS] Query [SEP] Document [SEP] → Transformer (BERT-like model) → Relevance score

Sort candidates by score → Send top-N to LLM

Typical Use Cases

- **Legal Search:** Rerank case laws matching a legal query.
- **Scientific Claim Verification:** Select the **most supportive evidence** for a claim.
- **FAQ/Customer Support:** Provide the **best single answer**, not just semantically related ones.
- **RAG Pipelines:** Step 1 = BM25/Bi-Encoder → Step 2 = Cross-Encoder reranking → LLM.



Pros and Cons



✓ Pros

Very high **precision**, fewer false positives.

Captures **fine-grained token interactions**.

Great for **reranking top-50 candidates**.

✗ Cons

Computationally expensive (per pair).

Not scalable to millions of docs alone.

Requires a good first-stage retriever.

Python code

```
from sentence_transformers import CrossEncoder
import numpy as np

# Load pretrained cross-encoder model
model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

# Query
query = "Who discovered penicillin?"

# Candidate documents (from BM25 or bi-encoder stage)
docs = [
    "Alexander Fleming discovered penicillin in 1928.",
    "Medical breakthroughs changed history.",
    "Penicillin is a widely used antibiotic."
]
```

```
# Prepare input pairs (query, doc)
pairs = [(query, doc) for doc in docs]

# Predict relevance scores
scores = model.predict(pairs)

# Sort documents by relevance
ranked_indices = np.argsort(scores)[::-1]
print(" ♦ Cross-Encoder Reranking Results:")
for idx in ranked_indices:
    print(f"- {docs[idx]} (score={scores[idx]:.4f})")
```

How it fits in a RAG pipeline

User Query → Stage 1: BM25 / Bi-Encoder → Top-50 docs



Stage 2: Cross-Encoder → Top-5 reranked docs



LLM with clean context → Final Answer

- This two-stage approach:
 - Bi-Encoder: Fast, scalable candidate generation.
 - Cross-Encoder: High-precision reranking.
 - Combined → Balanced retrieval (speed + accuracy).

Multi-Hop / Agentic Retrieval

- Multi-Hop Retrieval:
 - The system iteratively retrieves information in multiple steps ("hops"), using the output of one retrieval step to guide the next.
- Agentic Retrieval:
 - Extends multi-hop retrieval by allowing LLM agents to dynamically plan, call tools, and make reasoning decisions during retrieval (e.g., selecting which database, API, or search strategy to use next).
- Goal:
 - Answer complex, multi-fact questions that cannot be solved by looking at a single passage.

Why it's Needed

- Some queries require:
 - Combining facts from multiple documents (e.g., "Who is the spouse of the CEO of the company that owns Tesla?").
 - Chained reasoning (one fact leads to the next).
 - Dynamic knowledge gathering from structured and unstructured sources.
- A single-shot retriever (BM25, Bi-Encoder) may only fetch partial context, leading to incomplete or wrong answers.
- Multi-Hop/Agentic retrieval plans a chain of retrieval steps to gather all necessary evidence.

Architecture Flow

- Step 1: User Query → LLM Agent → Retrieval Call 1 → Doc Set 1
- Step 2: LLM Agent reads Doc Set 1 → generates new subquery
- Step 3: Retrieval Call 2 → Doc Set 2
- Step 4: Combine (Doc Set 1 + Doc Set 2) → Reasoning → Final Answer

The agent can:

- *Rephrase queries dynamically.*
- *Choose different retrieval tools (BM25, API, graph).*
- *Stop once enough information is collected.*

Examples

- **Question:** "Which university was attended by the author of 'The Theory of Relativity'?"
 - Hop 1: Retrieve → "Author of The Theory of Relativity is Albert Einstein."
 - Hop 2: Retrieve → "Albert Einstein attended the University of Zurich."
 - Answer: "University of Zurich."
- **Medical Reasoning:**
 - Hop 1: Find diagnosis details.
 - Hop 2: Retrieve latest treatment guidelines for that diagnosis.
- **Finance:**
 - Hop 1: Find company acquisition chain.
 - Hop 2: Retrieve CEO details from acquired company's parent.



Pros and Cons



Pros

Handles **complex, compositional queries**

Dynamic reasoning (agent can adapt queries/tools).

Combines evidence from **multiple sources**.

Reduces hallucinations by requiring **explicit evidence chain**.



Cons

Higher latency (multiple retrieval + reasoning steps).

Harder to implement (needs planning and tool orchestration).

Higher compute cost (more LLM + retrieval calls).

May propagate errors if first hop retrieves wrong info.

Python Demo (Simplified Multi-Hop Retrieval)

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")

docs = {
    "relativity": "The Theory of Relativity was written by Albert Einstein.",
    "einstein_university": "Albert Einstein attended the University of Zurich.",
}
```

```
# Step 1: Retrieve author of "Theory of Relativity"
query1 = "Who wrote The Theory of Relativity?"
embeddings = model.encode(list(docs.values()),
                            convert_to_tensor=True)

query1_emb = model.encode(query1,
                           convert_to_tensor=True)

scores1 = util.cos_sim(query1_emb, embeddings)[0]
first_doc = list(docs.values())[scores1.argmax()]
```

```
# Step 2: Generate new subquery based on first result
query2 = "Which university did Albert Einstein attend?"
query2_emb = model.encode(query2,
                           convert_to_tensor=True)

scores2 = util.cos_sim(query2_emb,
                        embeddings)[0]

second_doc = list(docs.values())[scores2.argmax()]
```

```
print(" Multi-Hop Retrieval Results:")
print( first_doc)
print( second_doc)
```



Agentic Retrieval Extension

When combined with LLM-based planning agents:

- Each step is guided by Chain-of-Thought reasoning inside the LLM.
- The agent can choose:
 - Which retriever to query (BM25, FAISS, API).
 - Whether to ask a clarifying question to user.
 - When to stop retrieval (sufficient evidence found).

Example frameworks:

- LangGraph Agents
- LlamaIndex Multi-Hop Retriever
- AutoGPT-like agentic patterns

When to Use Multi-Hop / Agentic Retrieval

- Queries that **require combining multiple facts**.
- Cases where **first retrieval results are ambiguous or incomplete**.
- Applications needing **explainability (evidence chains)**.
- When hallucinations are unacceptable (scientific, medical, legal domains).

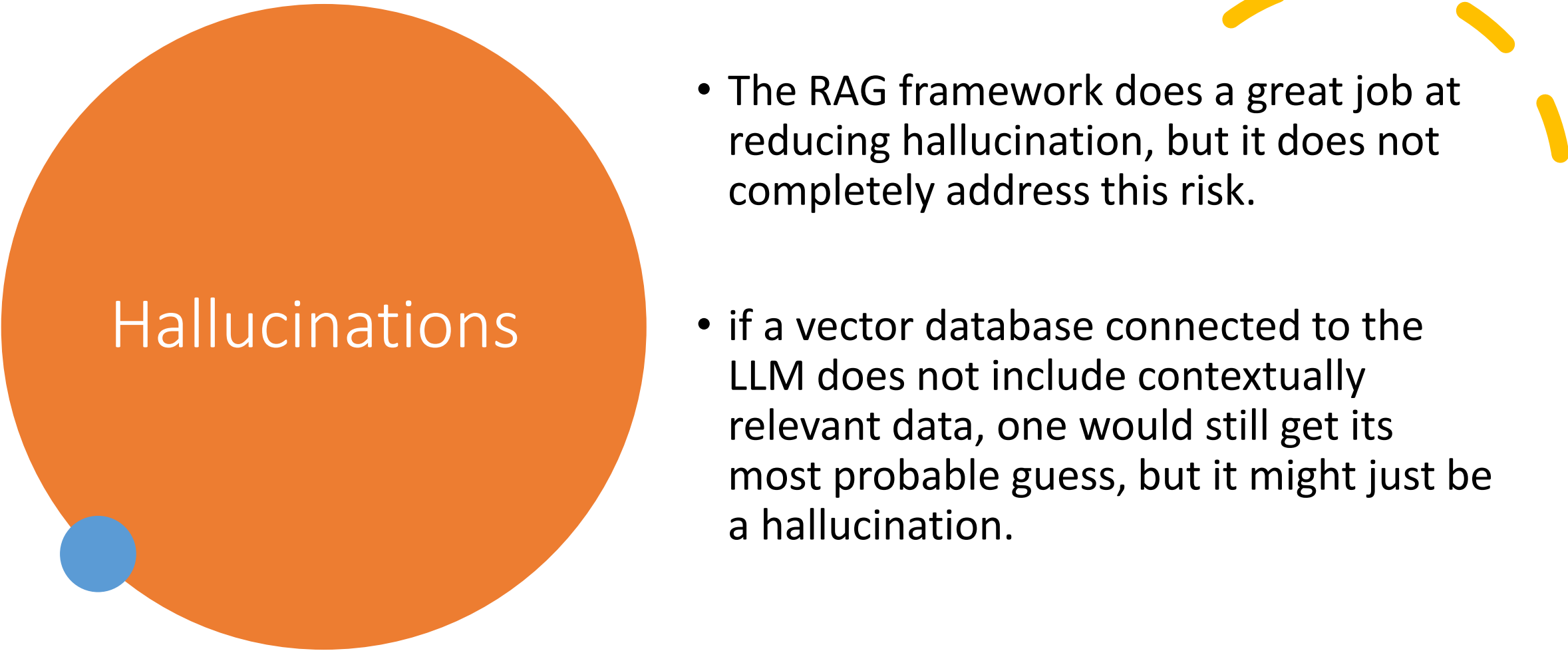


Summary (bi_encoder, cross-encoder, Agentic)

- Bi-Encoder / BM25: Single-hop retrieval → speed-oriented.
- Cross-Encoder: Reranking → accuracy boost.
- Multi-Hop / Agentic: Multi-step reasoning + retrieval → handles complex, compositional queries where one-shot search fails.

Challenges

Why we need RAG



Hallucinations

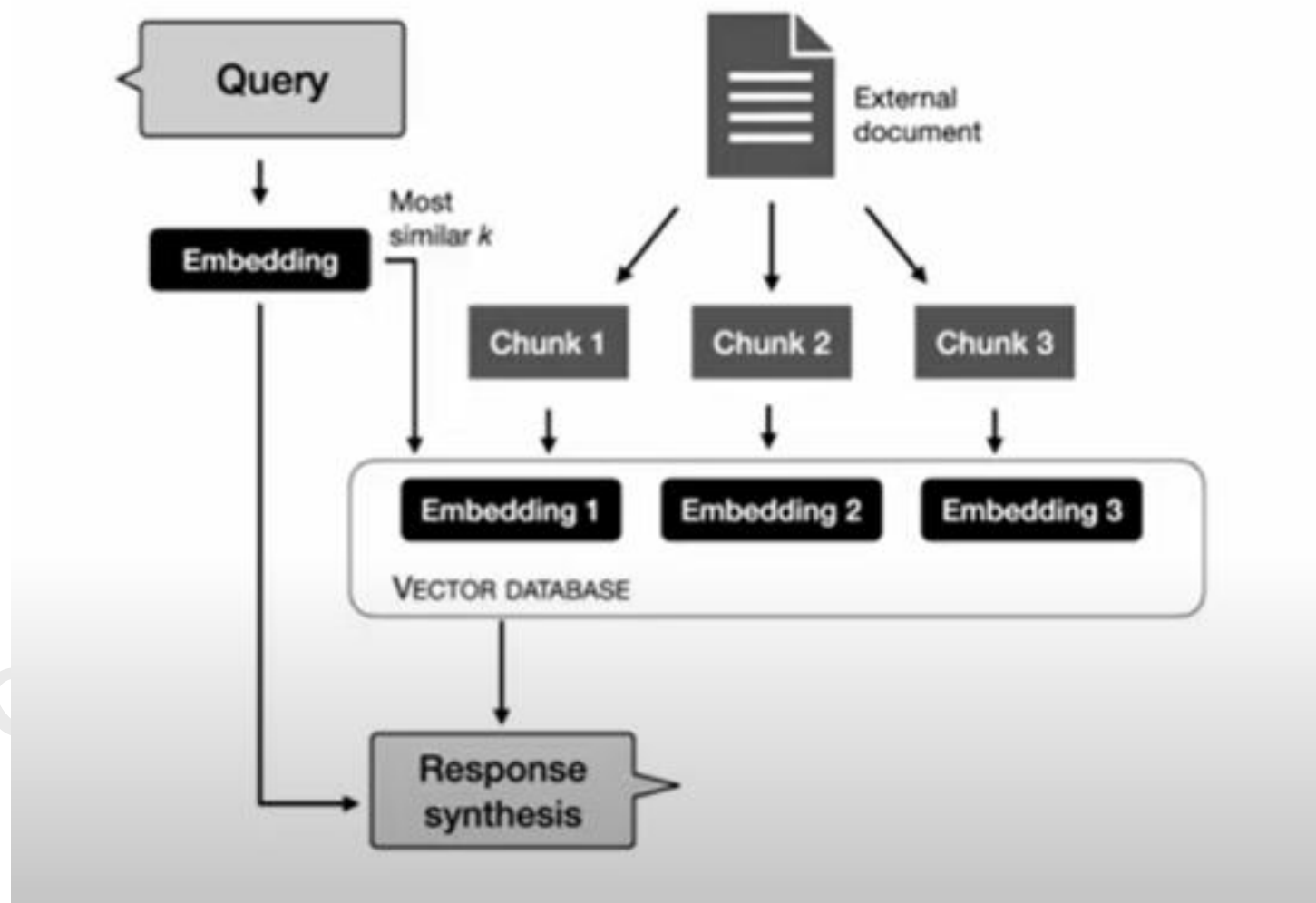
- The RAG framework does a great job at reducing hallucination, but it does not completely address this risk.
- if a vector database connected to the LLM does not include contextually relevant data, one would still get its most probable guess, but it might just be a hallucination.



Fine-tuning RAG models

- 
- often requires a specialized dataset that combines retrieval and generation tasks.
 - Annotating such data can be labor-intensive and expensive, as it involves not only providing appropriate retrieval passages but also generating coherent responses.

LLM-vector database approach



Data Quality

Use high-quality data sources:

- should be from reputable sources and
- should be carefully curated to ensure accuracy and consistency.

Clean and preprocess

- data should be cleaned to remove noise and inconsistencies.
- may involve removing duplicate entries, correcting spelling and grammatical errors, and normalizing text formats.

Monitor and update

- data should be monitored regularly to ensure that it remains accurate and up-to-date.
- involve updating the data with new information or removing outdated information.

Retrieval Effectiveness



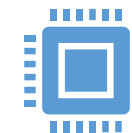
Use a robust retrieval algorithm:

The retrieval algorithm should be able to identify relevant passages even when the query is ambiguous or poorly formulated.



Consider using multiple retrieval methods:

Combining different retrieval methods, such as TF-IDF and BM25, can improve the overall effectiveness of the retrieval component.



Optimize retrieval parameters:

The parameters of the retrieval algorithm can be tuned to improve the relevance of the retrieved passages.

Computational Cost

Use efficient retrieval algorithms

There are a number of efficient retrieval algorithms that can be used to reduce the computational cost of RAG.

Use a lightweight language model

There are a number of lightweight language models that are available that can be used to reduce the computational cost of RAG.

Use hardware acceleration

Hardware acceleration, such as GPUs, can be used to further reduce the computational cost of RAG.



Lack of evaluation metrics

- Traditional evaluation metrics for natural language generation, such as BLEU or ROUGE, may not fully capture the effectiveness of retrieval-augmented generation systems.
- Developing appropriate evaluation methods that consider both retrieval and generation aspects is an ongoing challenge.

Backup slides

Created for training and reference purposes

Evaluating a QA/NLI Bot - RAG application



Question Answering (QA):

accuracy is a common metric used to evaluate the model's ability to correctly answer questions.

Example:

Suppose you have a dataset of question-answer pairs, and your model is trained to answer questions about a given passage.

True Positive (TP): The model correctly predicts the answer.

True Negative (TN): This might not be directly applicable in QA scenarios.

False Positive (FP): The model predicts an answer, but it's incorrect.

False Negative (FN): The model fails to predict an answer when it should.

Accuracy Formula:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions (TP + TN)}}{\text{Total Number of Predictions}}$$

For example, if your QA model correctly answers 80 out of 100 questions, the accuracy would be $80/100=0.8$ or 80%.

Machine Translation

- **BLEU (Bilingual Evaluation Understudy)**: Measures the overlap between the reference and machine-generated translations.
- **METEOR (Metric for Evaluation of Translation with Explicit ORdering)**: Considers synonyms and stemming in addition to exact word matches.
- **TER (Translation Edit Rate)**: Measures the number of edits required to change the machine translation to the reference translation.

BLEU (Bilingual Evaluation Understudy)



is a metric used to evaluate the quality of machine-generated translations by comparing them to one or more reference translations.



originally proposed for machine translation tasks, but it's also used in various natural language processing applications.



score ranges from 0 to 1,

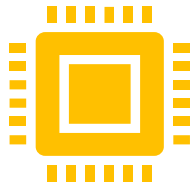


higher values indicating better translation quality.



computed based on the precision of n-grams (contiguous sequences of n items, usually words) in the generated translation compared to the reference translations.

explanation



N-gram Precision:

BLEU considers the precision of n-grams in the generated translation compared to the reference translations.

It evaluates how many n-grams in the generated translation match n-grams in the reference translations.



Brevity Penalty:

To avoid favoring overly short translations, BLEU includes a brevity penalty. It penalizes translations that are shorter than the reference translations.



BLEU Score Calculation:

The overall BLEU score is the geometric mean of the n-gram precisions, with the brevity penalty applied.

Example

Reference Translation:

- "The cat is on the mat."

Machine-Generated Translation:

- "The cat is sitting on the rug."

N-gram Precision Calculation - unigrams

- Count the number of unigrams (1-grams) in the machine-generated translation that appear in the reference translation:
 - "The" appears twice.
 - "cat" appears once.
 - "is" appears once.
 - "sitting" does not appear.
 - "on" appears once.
 - "the" appears once.
 - "rug" does not appear.

$$\text{Precision} = \frac{\text{Number of matching unigrams}}{\text{Total number of unigrams in the machine-generated translation}}$$

$$\text{Precision} = \frac{4}{7}$$

$$p_2 = \frac{\text{Number of matching bigrams}}{\text{Total number of bigrams in the machine-generated translation}}$$

$$p_2 = \frac{3}{6}$$

N-gram Precision Calculation for Bigrams (p_2):

- Count the number of bigrams in the machine-generated translation that appear in the reference translation:
 - "The cat" appears once.
 - "cat is" appears once.
 - "is sitting" does not appear.
 - "sitting on" does not appear.
 - "on the" appears once.
 - "the rug" does not appear.

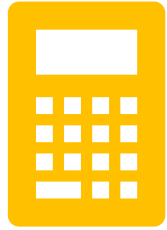
N-gram Precision Calculation for Trigrams (p3)

- Count the number of trigrams in the machine-generated translation that appear in the reference translation:
 - "The cat is" does not appear.
 - "cat is sitting" does not appear.
 - "is sitting on" does not appear.
 - "sitting on the" does not appear.
 - "on the rug" does not appear.

$$p_3 = \frac{\text{Number of matching trigrams}}{\text{Total number of trigrams in the machine-generated translation}}$$

$$p_3 = \frac{0}{5}$$

Brevity Penalty Calculation



Calculate the brevity penalty.



In this case, the machine-generated translation is longer than the reference, so the brevity penalty is not applied.



Brevity Penalty=1

calculate the BLEU score

- Given that:
 - $p_1=4/7$
 - $p_2=1/2$
 - $p_3=0$
- The brevity penalty is 1 because the machine-generated translation is longer than the reference.

$$\text{BLEU Score} = \text{Brevity Penalty} \times \exp\left(\frac{1}{3} \times (\log(p_1) + \log(p_2) + \log(p_3))\right)$$

$$\text{BLEU Score} = 1 \times \exp\left(\frac{1}{3} \times (\log\left(\frac{4}{7}\right) + \log\left(\frac{1}{2}\right) + \log(0))\right)$$

Since the logarithm of 0 is undefined, we need to handle this case. Conventionally, when a precision for any n-gram order is 0, the BLEU score is considered to be 0.

Therefore, the BLEU score in this case would be 0.

BLEU Score=0

choice of n-grams



BLEU-1 (unigrams): Evaluates the precision of individual words.



BLEU-2 (bigrams): Considers pairs of adjacent words.



BLEU-3 (trigrams): Considers sequences of three consecutive words.



BLEU-4 (quadgrams): Considers sequences of four consecutive words.



In practice, BLEU scores are often reported with a combination of these n -grams, such as BLEU-1, BLEU-2, BLEU-3, and BLEU-4, collectively referred to as BLEU-4.

Brevity Penalty (BP)

$$\begin{cases} 1 & \text{if } c \geq r \\ e^{(1-r/c)} & \text{if } c < r \end{cases}$$

In mathematical terms, this formula ensures that if the machine-generated translation is at least as long as the reference, there is no penalty (BP = 1).

If the machine-generated translation is shorter, the penalty is applied to adjust the BLEU score. The use of the exponential function $e^{(1-r/c)}$ emphasizes the penalty for significantly shorter translations.

- The Brevity Penalty (BP) is calculated based on the ratio of the length of the machine-generated translation (c) to the effective reference length (r).
- The effective reference length is the length of the reference translation (closest in length to the machine-generated translation).